

Arquitectura Orientada a Microservicios

Sebastián Vallejo

Ingeniería de Sistemas Mod. Virtual, Uniremington

Lenguaje de Programación Avanzado 2

Nixon Duarte Acosta

2026

CONTENIDO

Índice

1. Cuadro comparativo: monolito frente a microservicios

Diez criterios, cada uno anclado a un archivo verificable del repositorio

2. Ensayo breve: por qué las grandes empresas adoptan microservicios

El argumento organizativo, su costo real y la contratendencia

3. Diagrama de arquitectura

Contenedores, secuencia de la comunicación entre servicios y capas internas

4. Evidencia del código

Repositorio público, estructura y salidas capturadas del sistema en ejecución

5. Video de sustentación

Enlace a la grabación

ENTREGABLE 1

Cuadro comparativo

Comparación entre la arquitectura monolítica y la orientada a microservicios. Se solicitaban cinco criterios como mínimo; se desarrollan diez. La última columna no es decorativa: ancla cada criterio a un archivo concreto del repositorio, donde la diferencia puede verificarse ejecutando el código.

#	CRITERIO	ARQUITECTURA MONOLÍTICA	ARQUITECTURA DE MICROSERVICIOS	DÓNDE SE VE EN ESTE REPOSITORIO
1	Unidad de despliegue	Un solo artefacto. Cambiar una línea en el módulo de pedidos obliga a desplegar también usuarios, facturación y todo lo demás.	Cada servicio se despliega por separado y en su propio ritmo.	<code>make up-users</code> levanta Usuarios sin tocar Pedidos. Son dos imágenes Docker independientes, con dos <code>Dockerfile</code> distintos.
2	Escalabilidad	Se replica la aplicación completa, aunque el cuello de botella sea un solo módulo. Se paga por escalar código que no lo necesita.	Se replica solo el servicio saturado. El escalado sigue a la demanda real.	Cada servicio es una imagen con su propio proceso: replicar Pedidos (<code>--scale orders-service=3</code> , quitando antes el puerto fijo del host) no obliga a replicar Usuarios.
3	Base de datos	Una base compartida. La integridad referencial la garantiza el motor con claves foráneas y transacciones ACID.	Una base por servicio (<i>database per service</i>). Ningún servicio lee las tablas de otro.	<code>orders.user_id</code> no tiene FOREIGN KEY hacia <code>users</code> (<code>services/orders-service/db/schema.sql</code>): la tabla vive en otro contenedor, en otra red.
4	Acoplamiento e integridad	Acoplamiento en tiempo de compilación: llamada a función, con el compilador como red de seguridad.	Acoplamiento por contrato de red: la validación se sostiene llamando al otro servicio.	<code>CreateOrder</code> consulta el puerto <code>UserDirectory</code> antes de insertar (<code>services/orders-service/src/application/create-order.ts</code>).
5	Stack tecnológico	Uno solo para todo el sistema. Cambiar de lenguaje o de versión es una migración global.	Cada servicio elige el suyo según el problema que resuelve.	Usuarios en Python 3.13 + FastAPI ; Pedidos en TypeScript + Bun 1.4 . Conviven sin conocerse: solo comparten JSON sobre HTTP.
6	Aislamiento de fallos	Un fallo grave (fuga de memoria, excepción no controlada) derriba todo el proceso, incluidas las funciones sanas.	El fallo queda confinado al servicio afectado; el resto sigue operando, quizá degradado.	<code>make demo-fallo</code> : con Usuarios apagado, <code>POST /orders</code> responde 503 , pero <code>GET /orders</code> sigue devolviendo 200 .
7	Organización de los equipos (Ley de Conway)	Muchos equipos sobre un mismo código: conflictos de integración y una cola única de release.	Cada equipo es dueño de su servicio, de su base y de su despliegue.	Los dos servicios no comparten ni una línea de código: <code>services/users-service</code> y <code>services/orders-service</code> son mundos separados.
8	Complejidad operativa	Baja. Un proceso, un log, un despliegue; se depura con un depurador local.	Alta. Red, descubrimiento de servicios, trazas distribuidas, versionado de APIs, orquestación.	Para correr esto hacen falta 5 contenedores, 3 redes y 2 volúmenes (<code>compose.yaml</code>) donde un monolito usaría 1 proceso y 1 base. Ese es el precio.
9	Consistencia transaccional	<code>BEGIN ... COMMIT</code> abarca toda la operación de negocio, aunque toque varios módulos.	No hay transacción distribuida. Hay consistencia eventual, <i>sagas</i> y compensaciones.	Si Usuarios se cayera justo después de validar y antes de insertar, no habría <i>rollback</i> automático entre servicios: por eso <code>CreateOrder</code> valida antes de persistir.
10	Estrategia de pruebas	Las pruebas de integración son baratas: todo está en el mismo proceso.	Las unitarias siguen siendo baratas, pero probar la integración real exige levantar varios servicios o sustituirlos por dobles.	<code>make test</code> : 37 pruebas en milisegundos usando dobles (<code>tests/doubles.ts</code> , <code>tests/in_memory_user_repository.py</code>). Probar el sistema completo requiere <code>make up</code> .

Lectura del cuadro

Las diez filas se reducen a un solo intercambio:

Los microservicios ***cambian complejidad interna por complejidad operativa.***

Un monolito concentra la dificultad dentro del código: a medida que crece, los módulos se enredan y nadie se atreve a tocar el núcleo. Los microservicios extraen esa dificultad del código y la depositan en la infraestructura: el código de cada servicio queda pequeño y comprensible, pero aparecen problemas que antes no existían —latencia, fallos parciales, versionado de contratos, consistencia eventual.

El cambio conviene cuando la complejidad organizativa ya supera a la técnica: cuando el cuello de botella no es escribir el código, sino coordinar a la gente que lo escribe. Por eso es una arquitectura que rinde en empresas grandes y suele castigar a los equipos pequeños, que pagan el costo operativo sin tener el problema de coordinación que lo justifica.

Este repositorio hace visible ese precio: para dos servicios que en un monolito serían dos módulos y una llamada a función, aquí hacen falta cinco contenedores, tres redes, dos esquemas de base de datos, un adaptador HTTP con timeout y reintento, y una política explícita de qué hacer cuando el otro servicio no responde.

ENTREGABLE 2

Ensayo breve

¿Por qué las grandes empresas tecnológicas adoptan microservicios?

Cuando Amazon, Netflix o Uber migraron sus sistemas hacia microservicios, no lo hicieron porque la arquitectura fuera técnicamente superior. Lo hicieron porque su monolito había dejado de ser un problema de software para convertirse en un problema de organización. Entender esa distinción es lo que separa adoptar microservicios por criterio de adoptarlos por moda.

El primer motivo es organizativo, no técnico. La Ley de Conway observa que toda organización produce sistemas que replican su estructura de comunicación. Su corolario es incómodo: si dos mil ingenieros trabajan sobre un único artefacto desplegable, todos compiten por la misma cola de integración. El código no se vuelve más lento de escribir, sino más lento de *coordinar*. Amazon lo reconoció hacia 2002, cuando Jeff Bezos impuso que todos los equipos expusieran sus datos únicamente a través de interfaces de servicio: los "equipos de dos pizzas" obtuvieron fronteras técnicas que hacían imposible saltarse las fronteras de responsabilidad. Un microservicio es, antes que un patrón de diseño, un contrato de propiedad.

El segundo motivo es la frecuencia de despliegue. En un monolito la unidad mínima de cambio es la aplicación completa: corregir una errata en el módulo de notificaciones obliga a desplegar también el motor de pagos. Ese acoplamiento empuja a desplegar poco y en grandes lotes, lo que hace cada despliegue más peligroso y justifica desplegar aún menos: un círculo vicioso. Los microservicios lo rompen reduciendo el radio de impacto de cada cambio. Netflix, que hacia 2009 operaba un monolito Java, llegó a miles de despliegues diarios porque cada servicio podía liberarse sin sincronizarse con los demás.

El tercer motivo es económico. En un monolito la escalabilidad es uniforme: para absorber la carga de un módulo hay que replicar todo el proceso, pagando cómputo por código que nadie ejecuta. El perfil de carga real casi nunca es uniforme —en Uber, el emparejamiento de conductores y el procesamiento de pagos no crecen al mismo ritmo ni en el mismo horario—, y separar los servicios permite dimensionar cada uno según su demanda. A escala de miles de instancias, eso deja de ser una optimización y se vuelve una partida presupuestaria.

El cuarto motivo es la contención de fallos. En un proceso único, una fuga de memoria derriba también las funciones sanas. Al separar los servicios, el fallo queda confinado y el sistema se degrada en lugar de caerse: si el catálogo no responde, las recomendaciones se apagan pero el usuario sigue reproduciendo contenido. La contención no es automática —exige timeouts, reintentos acotados y cortacircuitos—, pero el monolito ni siquiera ofrece la posibilidad. El ejercicio práctico de este taller lo muestra: con el Servicio de Usuarios apagado, crear un pedido devuelve **503**, mientras que consultar los pedidos existentes sigue devolviendo **200**.

El quinto motivo es la libertad tecnológica. Cada servicio elige el lenguaje y el motor de datos adecuados a su problema, y migra de versión sin arrastrar al resto. Aquí el Servicio de Usuarios corre en Python con FastAPI y el de Pedidos en TypeScript sobre Bun, sin compartir una línea de código y sin que ninguno sepa en qué está escrito el otro.

Ahora bien, el precio es real y se paga por adelantado. Lo que era una llamada a función pasa a ser una llamada de red, con latencia y fallos parciales. Se pierde la transacción ACID y hay que sustituirla por consistencia eventual y compensaciones. Depurar deja de ser seguir una pila de llamadas y pasa a exigir trazas distribuidas. Por eso el consejo dominante entre quienes lideraron estas migraciones —Martin Fowler lo formuló como *monolith first*— es empezar por un monolito bien modularizado y extraer servicios solo cuando el dolor lo exija.

La evidencia más honesta de que no es una verdad universal la aportó la propia industria: en 2023 el equipo de Prime Video de Amazon reunió su sistema distribuido de monitoreo de audio y vídeo en un único servicio, reduciendo sus cos-

tos de infraestructura alrededor de un 90 %. Para ese caso, el tráfico entre componentes costaba más de lo que valía la separación.

La conclusión no es que los microservicios sean buenos o malos, sino que resuelven un problema específico: **permitir que muchos equipos avancen en paralelo sobre un sistema grande**. Las empresas que los adoptan con éxito tenían antes ese problema. Adoptarlos sin tenerlo significa pagar toda la complejidad operativa —redes, observabilidad, versionado de contratos, consistencia eventual— a cambio de un beneficio que no se necesita.

Referencias

- Conway, M. E. (1968). *How Do Committees Invent?* Datamation, 14(4), 28–31.
- Fowler, M. (2015). *MonolithFirst*. martinowler.com
- Newman, S. (2021). *Building Microservices* (2.^a ed.). O'Reilly Media.
- Richardson, C. *Microservices Patterns: Database per Service*. microservices.io
- Mauro, T. (2015). *Adopting Microservices at Netflix: Lessons for Architectural Design*. NGINX Blog.
- Amazon Prime Video Tech Blog (2023). *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%*.

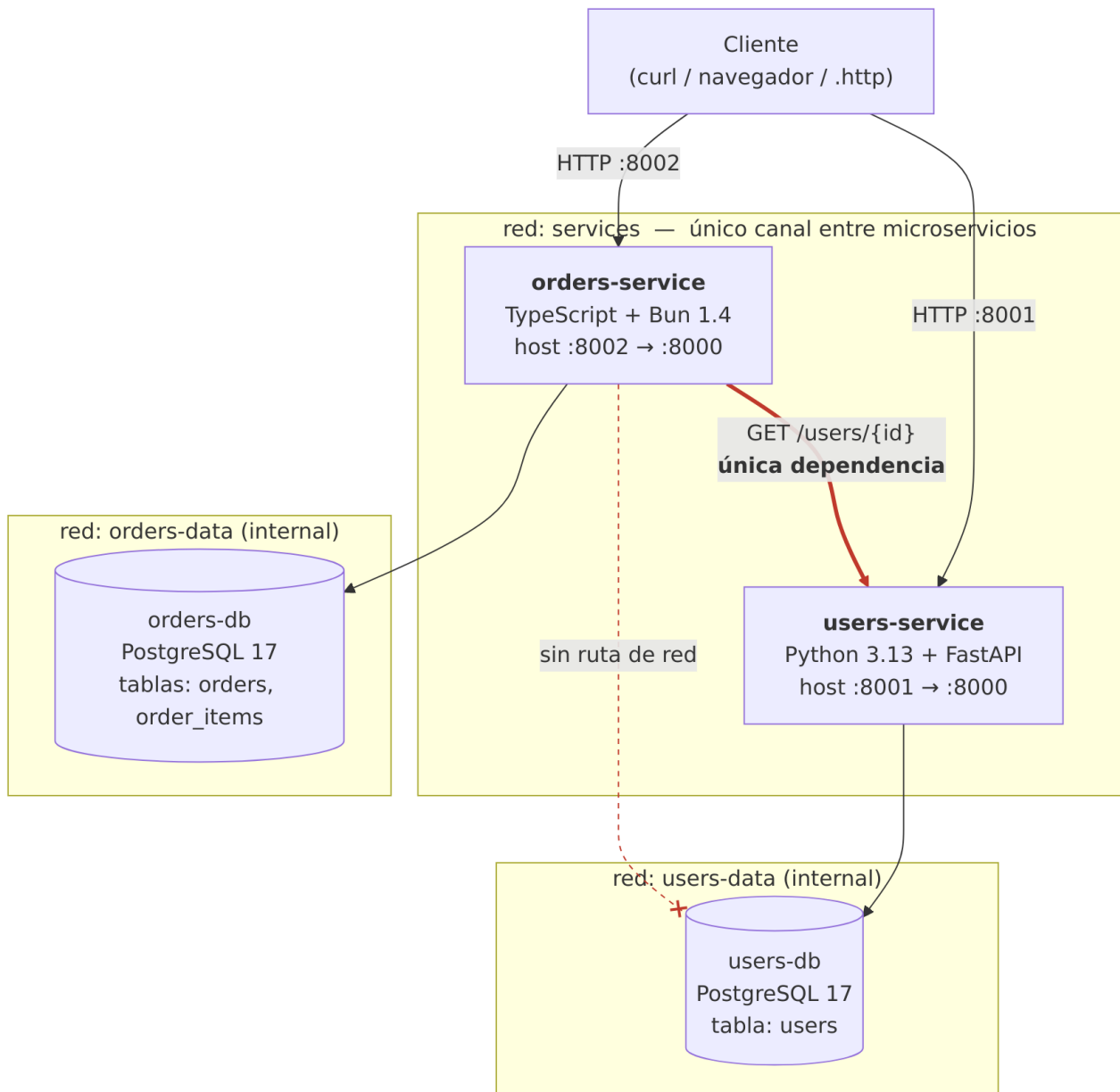
ENTREGABLE 4

Diagrama de arquitectura

Los diagramas están escritos en Mermaid dentro del repositorio, de modo que se versionan junto al código en lugar de vivir como imágenes sueltas.

1. Diagrama de contenedores

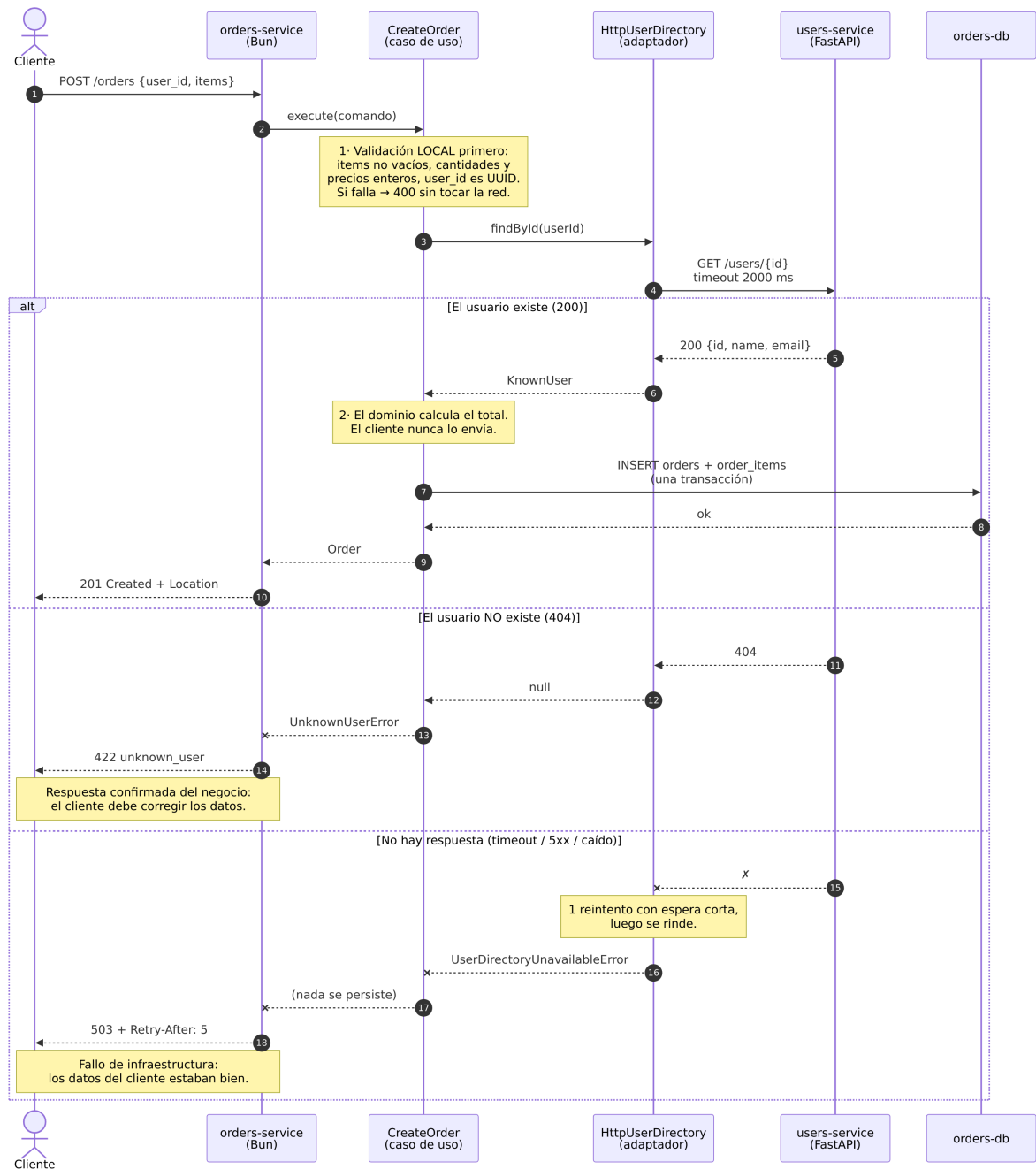
Lo que realmente se ejecuta al lanzar `make up`: cinco contenedores repartidos en tres redes.



La flecha punteada roja es el punto central del ejercicio: **no existe** y no puede existir. `orders-service` no está conectado a la red `users-data`, así que aunque alguien escribiera la cadena de conexión correcta, el paquete no llegaría. La frontera entre microservicios es física, no una convención de equipo.

2. Diagrama de secuencia: POST /orders

El recorrido completo de la única operación donde un servicio necesita algo que el otro posee. Incluye las tres ramas posibles.



Por qué 422 y 503 no pueden ser el mismo error

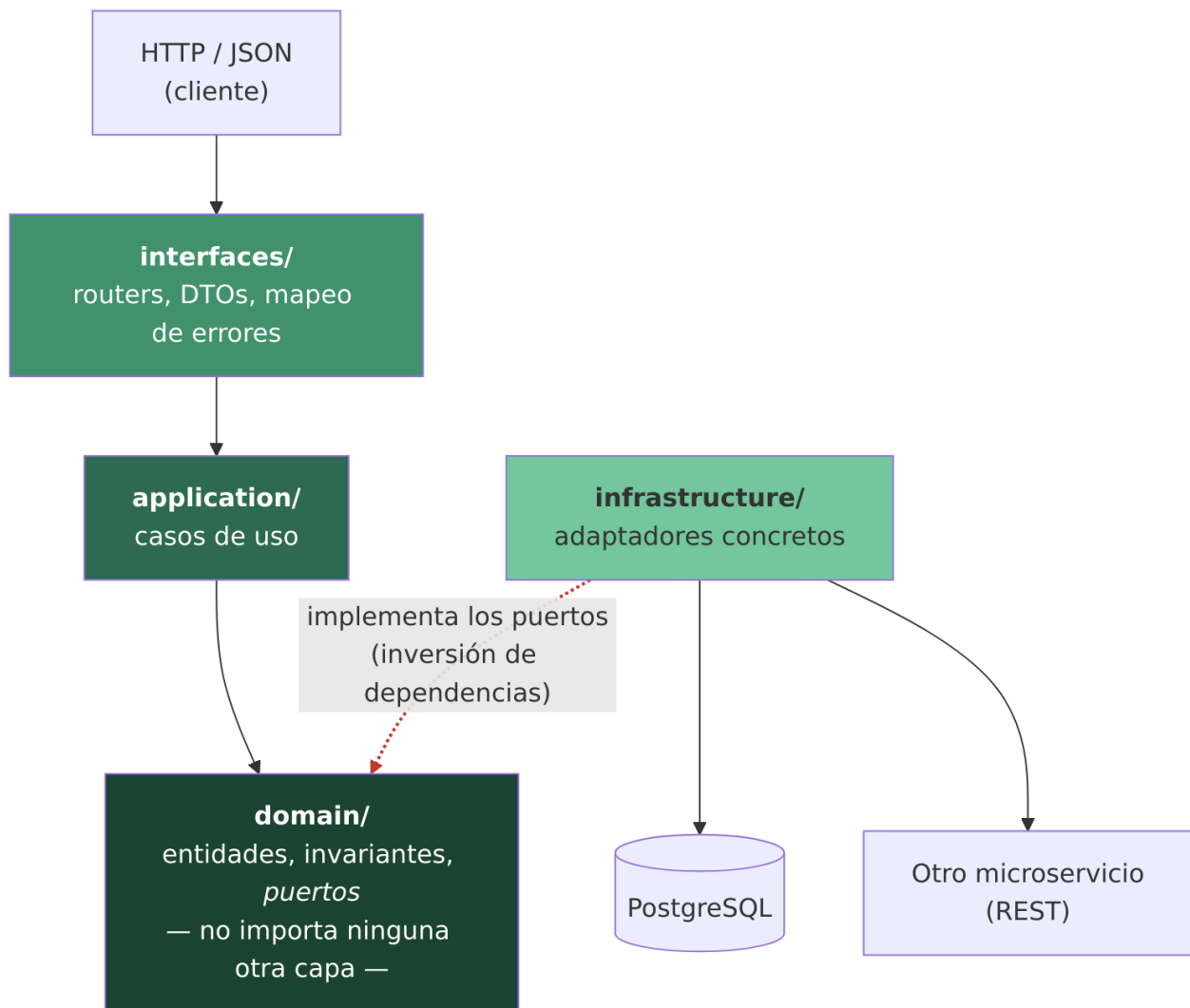
Es la decisión de diseño más importante del servicio y está codificada en dos excepciones distintas (`services/orders-service/src/domain/errors.ts`):

SITUACIÓN	LO QUE SABEMOS	ERROR DE DOMINIO	HTTP	QUÉ DEBE HACER EL CLIENTE
Usuarios responde 404	El usuario no existe	<code>UnknownUserError</code>	422	Corregir el <code>user_id</code>
Usuarios no responde	No sabemos si existe	<code>UserDirectoryUnavailableError</code>	503	Reintentar más tarde

Colapsar ambos casos en un 400 haría que el sistema rechazara pedidos de usuarios perfectamente válidos cada vez que la red tuviera un problema, y culpara al cliente de un fallo propio.

3. Arquitectura limpia: la regla de dependencias

Ambos servicios tienen **exactamente el mismo diagrama de capas**, aunque uno esté en Python y el otro en TypeScript. El contraste entre ellos es de stack, no de arquitectura.



La regla: las flechas apuntan siempre hacia adentro. **domain/** no importa nada de las otras capas; **infrastructure/** depende de **domain/** y nunca al revés. Por eso **I** apunta hacia **D**, aunque el flujo de datos en tiempo de ejecución vaya en sentido contrario: eso es la **inversión de dependencias**.

Cómo se comprueba que la regla se cumple. No hace falta creer en el diagrama: si el dominio dependiera de Postgres o de `fetch`, sería imposible ejecutar sus pruebas sin levantar infraestructura. `make test` corre 37 pruebas —incluida "¿qué pasa si el otro servicio se cae?"— en milisegundos, sin base de datos, sin red y sin Docker Compose.

Puertos y adaptadores

Un **puerto** es una interfaz declarada por el dominio; un **adaptador** es su implementación concreta, elegida en el arranque. La última fila es la que hace posible probar el fallo del otro servicio sin apagar nada.

SERVICIO	PUERTO (EN <code>PORTALITY</code>)	ADAPTADOR DE PRODUCCIÓN	ADAPTADOR EN PRUEBAS
Usuarios	<code>UserRepository</code>	<code>PostgresUserRepository</code>	<code>InMemoryUserRepository</code>
Pedidos	<code>OrderRepository</code>	<code>PostgresOrderRepository</code>	<code>InMemoryOrderRepository</code>
Pedidos	<code>UserDirectory</code>	<code>HttpUserDirectory</code> → REST	<code>FakeUserDirectory</code> , <code>EmptyUserDirectory</code> , <code>BrokenUserDirectory</code>

El *composition root* —el único punto donde se decide qué adaptador se usa— es `interfaces/http/container.py` en Usuarios y `interfaces/http/container.ts` en Pedidos. Cambiar de motor de persistencia es editar una línea en ese archivo.

ENTREGABLE 3

Evidencia del código

El código completo, con su historial de commits, está publicado en un repositorio público de GitHub:

<https://github.com/svallejo-dev/lpa-two-microservices>

Se levanta con tres comandos, y el único requisito es Docker: `make doctor`, `make up`, `make demo`.

Qué contiene

RUTA	CONTENIDO
<code>services/users-service/</code>	Servicio de Usuarios — Python 3.13 + FastAPI + PostgreSQL
<code>services/orders-service/</code>	Servicio de Pedidos — TypeScript + Bun 1.4 + PostgreSQL
<code>services/orders-service/openapi.yaml</code>	Contrato OpenAPI 3.1 escrito a mano (<i>API Design First</i>)
<code>compose.yaml</code>	Cinco contenedores, tres redes, dos volúmenes
<code>Makefile</code>	Punto de entrada único: levantar, probar, demostrar
<code>docs/</code>	Cuadro comparativo, ensayo, arquitectura y contrato
<code>scripts/</code>	<code>demo.sh</code> y <code>demo-fallo.sh</code> para la sustentación

Ambos servicios comparten el mismo diseño interno

Las cuatro capas son idénticas en los dos microservicios, aunque uno esté escrito en Python y el otro en TypeScript. El contraste entre ellos es de tecnología, no de arquitectura:

```
src/<servicio>/
├─ domain/          entidades, invariantes y PUERTOS (cero dependencias externas)
├─ application/     casos de uso; orquestan el dominio a través de los puertos
├─ infrastructure/  ADAPTADORES: PostgreSQL, cliente HTTP, configuración
└─ interfaces/http/ rutas, DTOs y traducción de errores de dominio a HTTP
```

El punto de comunicación entre los dos servicios

Este es el caso de uso donde un microservicio necesita algo que otro posee. Depende del *puerto* `UserDirectory`, no de `fetch` ni de una URL: por eso puede probarse sin red.

```
async execute(command: CreateOrderCommand): Promise<Order> {
  // 1. Validación LOCAL primero. Si el pedido está mal formado, se rechaza
  // sin gastar una llamada de red al otro servicio.
  const order = Order.place(command.userId, command.items);

  // 2. Validación REMOTA. Como las bases de datos están separadas, no existe
  // una foreign key que garantice que el usuario existe: hay que
  // preguntarlo. Si el directorio no responde, este `await` lanza
  // UserDirectoryUnavailableError y el pedido NO se persiste.
  const user = await this.users.findById(order.userId);
  if (user === null) {
    throw new UnknownUserError(order.userId);
  }

  // 3. Persistencia. Solo se llega aquí con un usuario confirmado.
  await this.orders.add(order);
  return order;
}
```

Evidencia de ejecución

Las siguientes salidas se capturaron del sistema en funcionamiento, no están transcritas a mano.

make up — cinco contenedores, todos saludables

```
SERVICE      STATUS
orders-db     Up 40 minutes (healthy)
orders-service Up 40 minutes (healthy)
users-db      Up 40 minutes (healthy)
users-service Up 40 minutes (healthy)
```

make test — 37 pruebas de dominio, casos de uso y contrato, sin base de datos ni red

```
==> Pruebas: Servicio de Usuarios
..... [100%]
==> Pruebas: Servicio de Pedidos
21 pass
0 fail
```

make demo — flujo completo entre los dos microservicios

```
0. Estado de las dos dependencias del Servicio de Pedidos
GET http://localhost:8002/health/ready
{"status":"ready","service":"orders-service","dependencies":{"database":"ok","users_service":"ok"}}
[HTTP 200]

1. Crear un usuario en el Servicio de Usuarios (Python + FastAPI)
{"id":"7ca2edfd-8f94-44e0-993f-40bdc9ced8a1","name":"Estudiante
LPA2","email":"demo-1789103458@example.com","created_at":"2026-09-11T05:10:58.360924Z"}
id del usuario: 7ca2edfd-8f94-44e0-993f-40bdc9ced8a1

2. Crear un pedido en el Servicio de Pedidos (TypeScript + Bun)
Pedidos NO tiene la tabla de usuarios: consulta a Usuarios por HTTP antes de insertar.
POST http://localhost:8002/orders
{"id":"6a42510c-1199-4f39-a9ce-93d979490b0e","user_id":"7ca2edfd-8f94-44e0-993f-40bdc9ced8a1","status":"PENDIENTE","total":
54000,"items":[{"sku":"TECLADO","quantity":2,"unit_price":15000,"subtotal":30000},
{"sku":"MOUSE","quantity":3,"unit_price":8000,"subtotal":
24000}], "created_at":"2026-09-11T05:10:58.396Z"}
[HTTP 201]
El total (54000) lo calculo el dominio; el cliente nunca lo envia.

3. Consultar los pedidos de ese usuario
GET http://localhost:8002/orders?user_id=7ca2edfd-8f94-44e0-993f-40bdc9ced8a1
[{"id":"6a42510c-1199-4f39-a9ce-93d979490b0e","user_id":"7ca2edfd-8f94-44e0-993f-40bdc9ced8a1","status":"PENDIENTE","total":
54000,"items":[{"sku":"TECLADO","quantity":2,"unit_price":15000,"subtotal":30000},
{"sku":"MOUSE","quantity":3,"unit_price":8000,"subtotal":
24000}], "created_at":"2026-09-11T05:10:58.396Z"}]
[HTTP 200]

4. Intentar un pedido para un usuario que no existe
Usuarios responde 404 -> Pedidos lo traduce a 422: los datos del cliente estan mal.
POST http://localhost:8002/orders
{"error":"unknown_user","message":"El usuario '00000000-0000-0000-0000-000000000000' no existe en
el Servicio de Usuarios."}
[HTTP 422]

5. Intentar un pedido invalido (sin lineas)
Se rechaza en el dominio, sin gastar una llamada de red al otro servicio.
POST http://localhost:8002/orders
{"error":"empty_order","message":"Un pedido debe tener al menos una linea."}
[HTTP 400]

Listo.
Para ver que ocurre cuando el Servicio de Usuarios se cae: make demo-fallo
```

make demo-fallo — comportamiento con el Servicio de Usuarios caído

```
0. Preparacion: un usuario que SI existe
   id: 255e57c5-9dd2-4def-8743-cafed30aed1c (este usuario es real, el pedido deberia poder crearse)

1. Apagando el Servicio de Usuarios...
   Servicio de Usuarios DETENIDO.

2. Intentando crear un pedido de ese mismo usuario valido
   HTTP/1.1 503 Service Unavailable
   retry-after: 5
   {"error":"user_directory_unavailable","message":"No se pudo verificar el usuario porque el Servicio
de Usuarios no respondio (The operation timed out.). El pedido no fue creado; reintente mas tarde."}
   Tardo 5 s: el timeout acota la espera en lugar de colgar la peticion.
   503, no 400: el problema no son los datos del cliente. Y el pedido NO se guardo.

3. Lo que SIGUE funcionando mientras Usuarios esta caido
   GET /orders -> [HTTP 200]
   Las lecturas de pedidos no dependen de Usuarios: el fallo esta acotado, no tumba el servicio entero.

4. Diagnostico automatico
   {"status":"degraded","service":"orders-service","dependencies":{"databa-
se":"ok","users_service":"unreachable"}}
   La sonda dice exactamente cual de las dos dependencias fallo.

5. Restaurando el Servicio de Usuarios...
   Servicio de Usuarios ARRIBA.

6. El mismo pedido, ahora sin cambiar nada en el cliente
   {"id":"12d2d661-460a-4ba7-aa56-c6a7d99600f9","user_id":"255e57c5-9dd2-4def-8743-cafed30ae-
d1c","status":"PENDIENTE","total":250000,"items":[{"sku":"SSD","quantity":1,"unit_price":
250000,"subtotal":250000}],"created_at":"2026-09-11T05:11:06.800Z"}
   [HTTP 201]
   Recuperacion automatica: no hubo que reiniciar Pedidos ni limpiar datos a medias.
```

ENTREGABLE 5

Video de sustentación

La sustentación recorre el sistema en funcionamiento siguiendo el mismo guion de `make demo` y `make demo-fallo`: creación de un usuario, creación de un pedido con la validación entre servicios, los errores de negocio, y el comportamiento del sistema cuando el Servicio de Usuarios se cae.

Enlace del video de sustentación

Para fijarlo de forma definitiva, escriba la URL en la constante `VIDEO_URL` de `scripts/build-pdf.py` y vuelva a ejecutar `make pdf`.